

Arhitectura calculatoarelor

Curs 2

2. Proiectarea Sistemelor Digitale

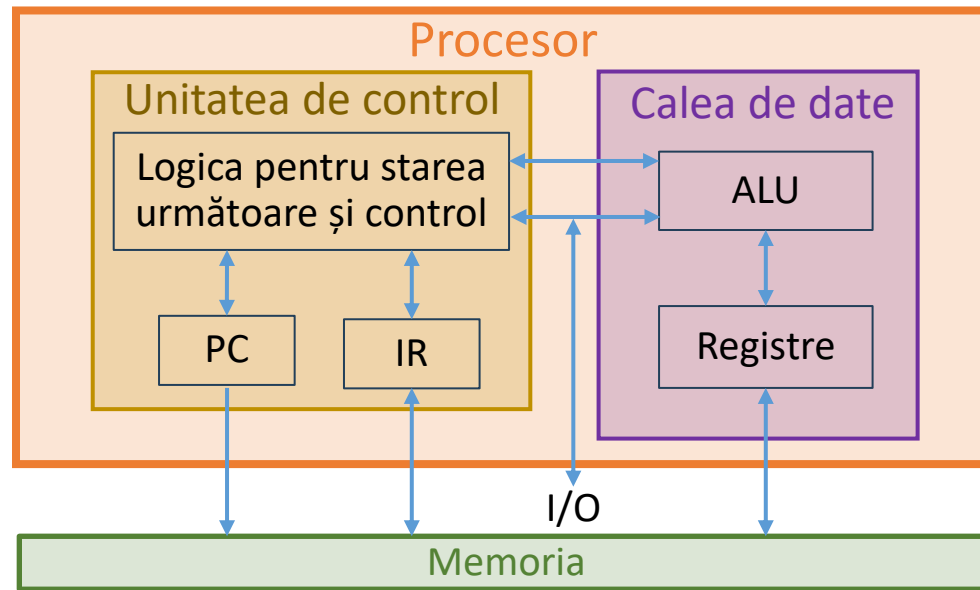
Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

Obiective

1. Register Transfer Level (RTL)
2. Sinteza la nivel înalt

2.1 Register Transfer Level (RTL)



- Un sistem digital (procesor) conține:
 - Unitatea de control (Control Unit) – determină secvența de operații aplicate datelor din calea de date
 - Calea de date (Datapath) – logica de procesare, registre, magistrale de transfer a datelor.

2.1 Register Transfer Level (RTL)

Definiție: RTL = notație algebrică pentru descrierea unui sistem prin operații între elementele sale de memorare:

$$\text{Ex: } R_{\text{dest}} \leftarrow f(R_{\text{src1}}, R_{\text{src2}}, \dots R_{\text{srcN}})$$

- Există 2 niveluri de descriere:
 - **RTL abstract:** descriere comportamentală
 - Nu ține cont de structură și constrângeri de timp
 - **RTL concret:** descrie implementarea RTL abstract
 - Ține cont de structură și de constrângerile de timp
- **Obs:** RTL **nu** se execută, ci descrie funcționarea.

Definiție: Metoda de proiectare RTL = conversia RTL abstract
→ RTL concret

2.1 Register Transfer Level (RTL)

- RTL concret include:
 1. Setul de registre;
 2. Unitățile Funcționale (UF) - realizează operațiile pe registre și memorie;
 3. Unitatea de control - generează secvența de operații.
- Elemente de descriere RTL:
 1. Micro-operație: $X \leftarrow Y + Z$
 2. Propoziție (statement): $(\text{condiție}) \rightarrow X \leftarrow Y + Z$

Notăția RTL = **Register Transfer Notation** (RTN)

- nu este standardizată

2.1 Register Transfer Level (RTL)

Exemplu RTN

$\leftarrow$	Atribuire
$=, \neq$	Test pentru egalitate / inegalitate
$ $	Concatenare pentru șiruri de biți
$X \leftarrow Y$	Transfer de date din registrul Y în registrul X
$X \leftarrow 0$	Încarcă registrul X cu valoarea 0
$X \leftarrow Y + Z$	Adună registrul Y cu registrul Z și încarcă rezultatul în X
$X \leftarrow Y \vee Z$	SAU între Y și Z; încarcă rezultatul în registrul X
$DR \leftarrow M[AR]$	Încarcă în DR conținutul memoriei de la adresa dată de AR
$R1 \leftarrow \gg R1$	Deplasare logică la dreapta cu o poziție pentru registrul R1
$R2 \leftarrow \ll R2$	Deplasare logică la stânga cu o poziție pentru registrul R2
$X \leftarrow Y, A \leftarrow B$	Transferuri paralele (concurente)
$(cond) \rightarrow A \leftarrow B$	Dacă cond = 1, atunci copiază B în A
$S0 \rightarrow A \leftarrow B$	Dacă starea curentă este S0, atunci copiază B în A
$P \wedge (a \wedge b) \rightarrow R2 \leftarrow R3$	Dacă starea curentă este P, și dacă a și b este adevărată atunci copiază R3 în R2 (a și b sunt semnale)

2.2 Sinteza la nivel înalt (HLS)

2.2.1 Definiție – Sinteza la nivel înalt = High Level Synthesis (HLS)

- Metodologie de implementare **automatizată** a unei descrieri comportamentale (la nivel de algoritm), cu registre, memorii, unități de calcul și control;
- Respectă constrângeri de timp, resurse folosite, consum.

Pași de implementare a sistemelor de calcul:

1. Specificațiile sistemului;
 2. Descrierea comportamentală;
 3. Descrierea RTL;
 4. Implementarea circuitului;
 5. Implementarea fizică;
 6. Testare și producție.
- HLS {

2.2 Sinteza la nivel înalt (HLS)

2.2.2 Etapele HLS

1. Alocarea resurselor – stabilirea componentelor hardware necesare;
2. Planificare (scheduling) – alocarea operațiilor în timp;
3. Legarea modulelor (binding) – asocierea operațiilor la UF;
4. Sinteza căii de date – implementarea cu UF, registre, conexiuni;
5. Sinteza unității de control – implementarea și stabilirea tipului unității de control și a regimului de folosire a semnalului de ceas.

2.2 Sinteza la nivel înalt (HLS)

2.2.3 Proprietăți ale HLS

2.2.3.1 Calcul orientat pe flux de date – folosește **graf de flux de date** – **Data-Flow Graph (DFG)** compus din:

- **Noduri** – reprezintă operațiile;
- **Arce** – reprezintă relațiile de precedență.

Ex: $\text{res} = (A+B) \times [(A+B) / (C-D)] - [(A+B) / (C-D) + E \times F]$

Obs: HLS poate realiza planificare **concurrentă**:

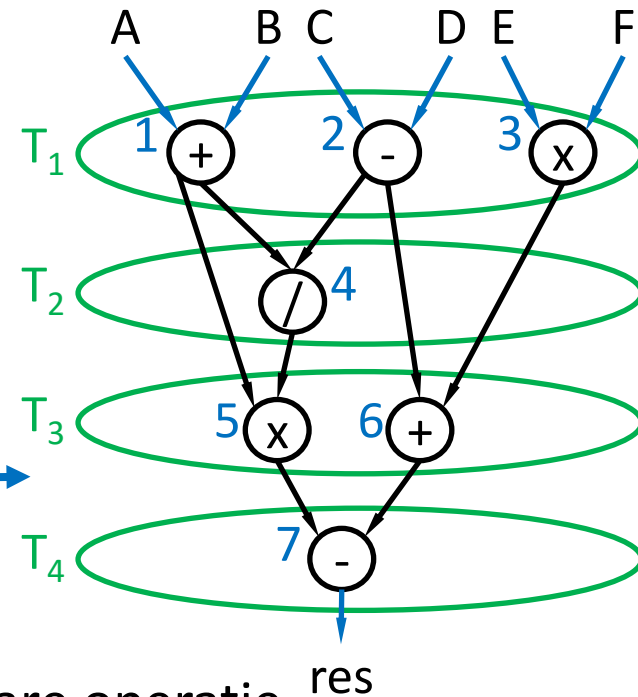
T_1 : 1. $\text{tmp1} = A + B$; 2. $\text{tmp2} = C - D$; 3. $\text{tmp3} = E \times F$

T_2 : 4. $\text{tmp4} = \text{tmp1} / \text{tmp2}$

T_3 : 5. $\text{tmp5} = \text{tmp1} \times \text{tmp4}$; 6. $\text{tmp6} = \text{tmp4} + \text{tmp3}$

T_4 : 7. $\text{res} = \text{tmp5} - \text{tmp6}$

DFG



2.2.3.2 Atribuire unică – 2 intrări + 1 ieșire la fiecare operație

- **Notă:** Fiecare rezultat se scrie într-o variabilă unică.

Ex: $x = a + b$

$x1 = a + b$

$y = a \times c$

$y = a \times c$

$z = x + d$

$z = x1 + d$

$x = y - d$ (x devine x1, x2, x3)

$x2 = y - d$

$x = x + c$

$x3 = x2 + c$

2.2 Sinteza la nivel înalt (HLS)

2.2.3.3 Compilare – face optimizare comportamentală (nu la nivel hardware):

1. Reducerea înălțimii grafului



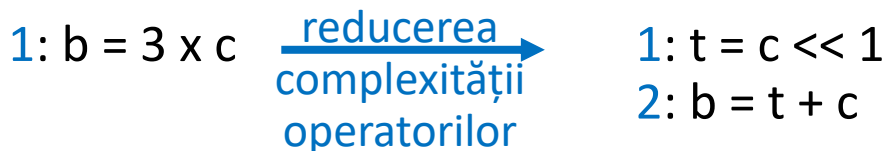
2. Propagarea constantelor și a variabilelor



3. Eliminarea sub-expresiilor comune



4. Reducerea complexității operatorilor



5. Rearanjarea codului



2.2 Sinteza la nivel înalt (HLS)

Exemplu

repeat

$x_l = x + dx;$

$u_l = u - (3 \cdot x \cdot u \cdot dx) - (3 \cdot y \cdot dx);$

$y_l = y + u \cdot dx;$

$c = x_l < a;$

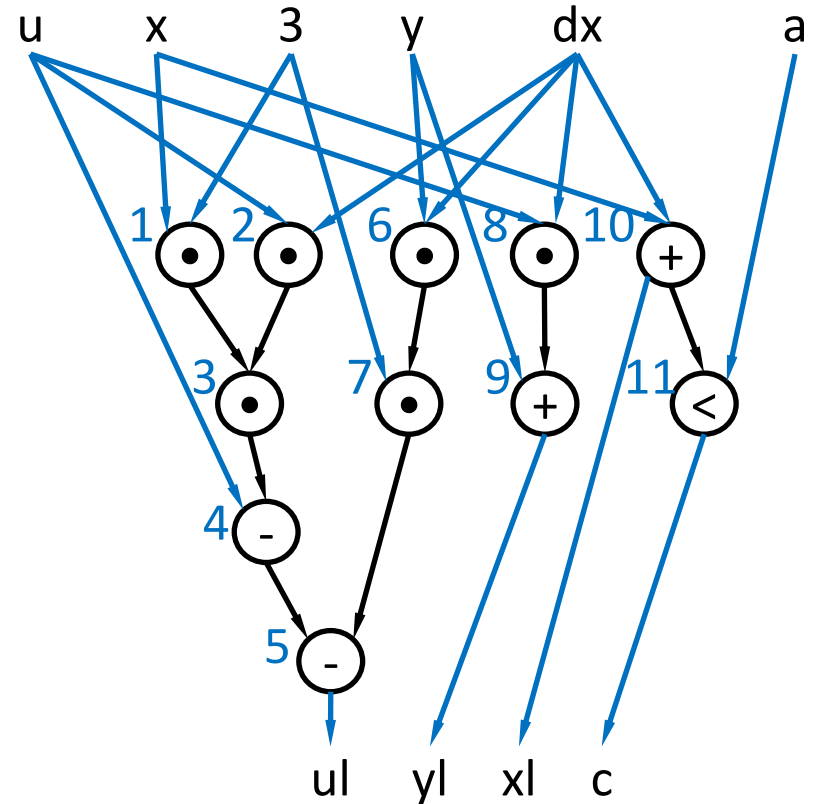
$x = x_l; u = u_l; y = y_l;$

until (c);

1. Alocarea resurselor

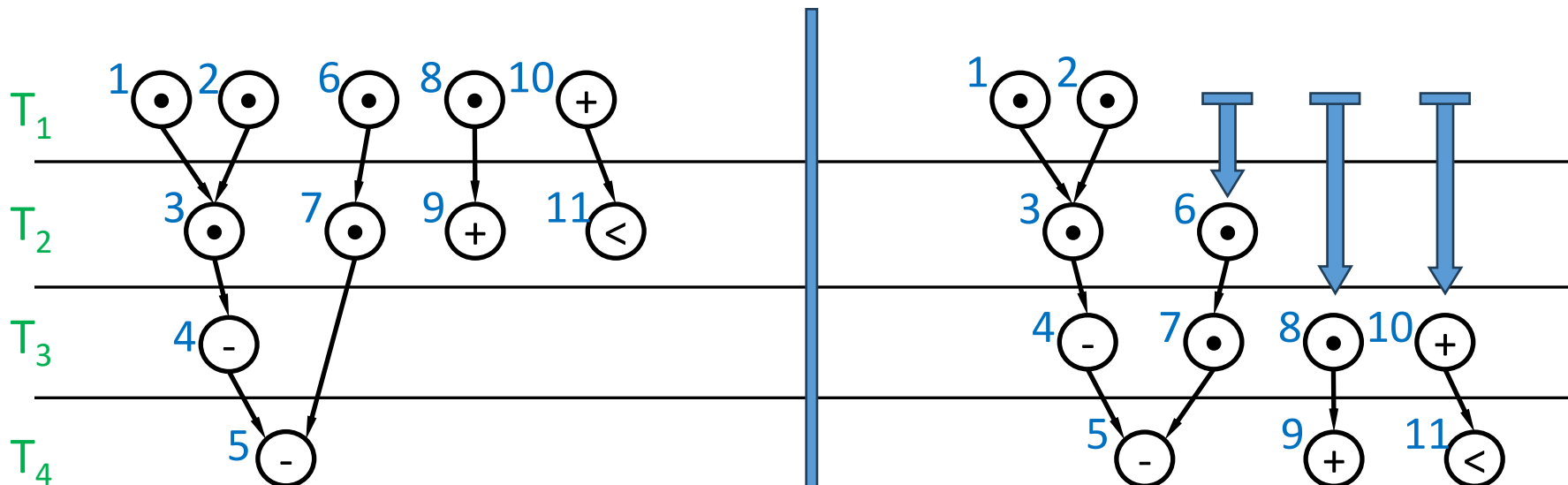
Alocarea nodurilor în DFG

Instrucțiune	Noduri
$x_l = x + dx$	10
$u_l = u - (3 \cdot x \cdot u \cdot dx) - (3 \cdot y \cdot dx)$	1-7
$y_l = y + u \cdot dx$	8, 9
$c = x_l < a$	11



2.2 Sinteza la nivel înalt (HLS)

2.A Planificare (scheduling) **fără constrângeri** de resurse/UF



As Soon As Possible (ASAP)

- operațiile încep mediat ce este posibil

As Late As Possible (ALAP)

- operațiile încep cu maxim de întârziere

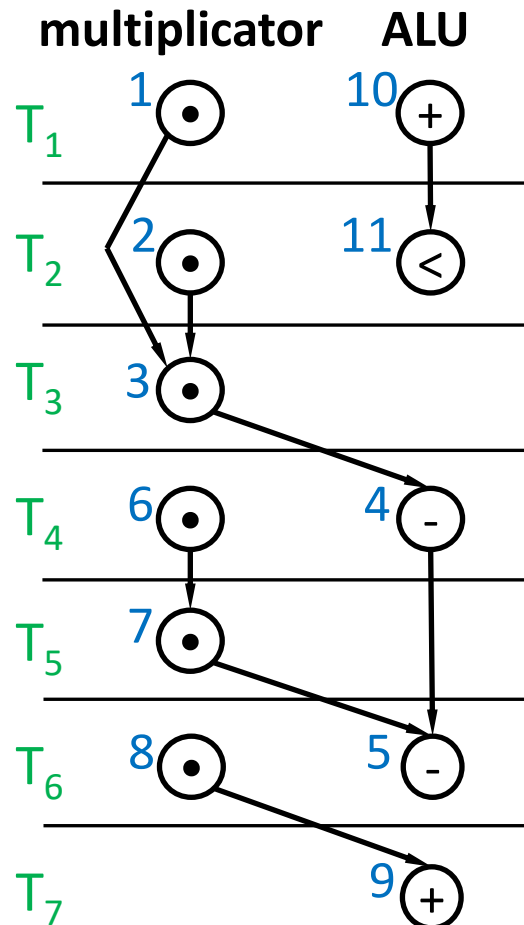
Mobilitatea = intervalul de start pentru o operație.

- Determinare: se planifică ASAP și ALAP și se calculează diferența de start.
- Operațiile cu mobilitate 0 alcătuiesc **calea critică** (cea mai lungă).
- Este: 0 pt. operațiile 1-5; 1 pt. operațiile 6-7; 2 pt. operațiile 8-11.

2.2 Sinteza la nivel înalt (HLS)

2.B Planificare (scheduling) **cu constrângeri** de resurse/UF

- Ex: Avem la dispoziție 1 multiplicator (•) și 1 ALU (+, -, <).

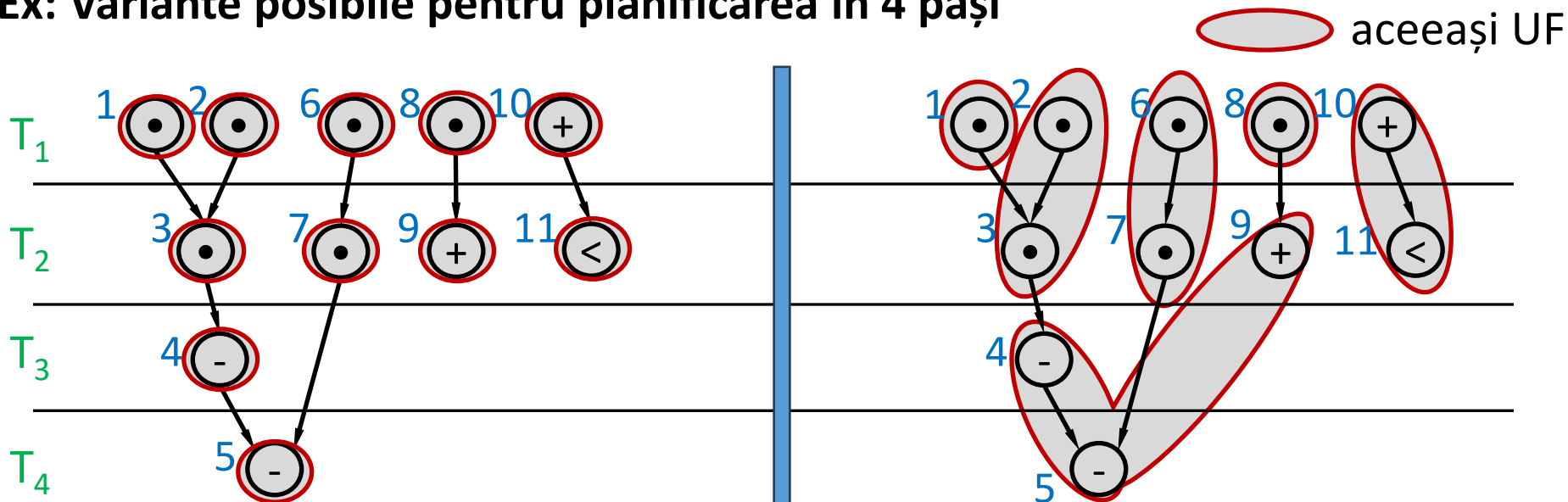


DFG cu 1 multiplicator și 1 ALU

2.2 Sinteza la nivel înalt (HLS)

3. Legarea modulelor (binding)

Ex: Variante posibile pentru planificarea în 4 pași



Maxim UF: 6 înmulțitoare + 5 ALU

Notă: Câte o resursă pentru fiecare operație.

Minim UF: 4 înmulțitoare + 2 ALU

Notă: **Refolosirea resurselor** (resource sharing) = utilizarea aceleiași componente pentru mai multe operații, la momente diferite.

2.2 Sinteza la nivel înalt (HLS)

4. Sinteza căilor de date

- **Definiție:** Reprezintă procesul de generare a căilor de date pentru DFG.
 - Componente hardware în calea de date: bloc de registre, memorii, multiplexoare.
 - Pași de sinteză:
 - **Alocare** (allocation): selectarea registrelor și a celor mai potrivite componente;
 - **Legare** (binding): asocierea optimală a operațiilor la unități funcționale (UF) și a variabilelor la registre.
- **Algoritmul de legare a registrelor** (register binding):
1. Analizează timpul de viață al variabilelor (cicluri de ceas în care variabila este necesară);
 2. Stabilește numărul de registre.

2.2 Sinteza la nivel înalt (HLS)

5.A Sinteza unității de control – la varianta **cu refolosirea resurselor**

- Se folosește un automat cu stări finite (Finite State Machine – FSM) pentru implementarea fluxului descris la planificare.
- 1 stare la fiecare impuls de ceas.
- FSM coordonează semnalele de control pentru toate componentele: MUX-uri, registre, memorii, ALU.

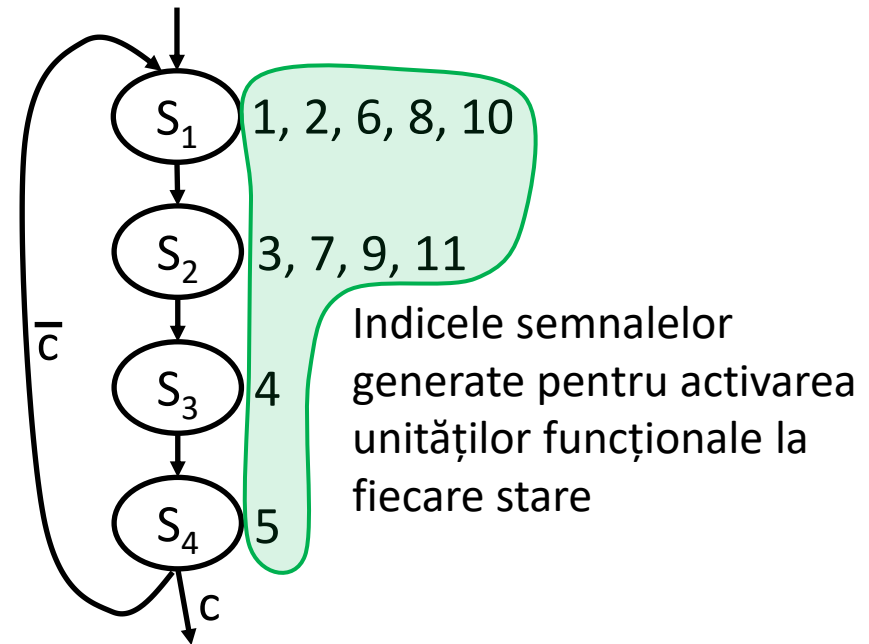
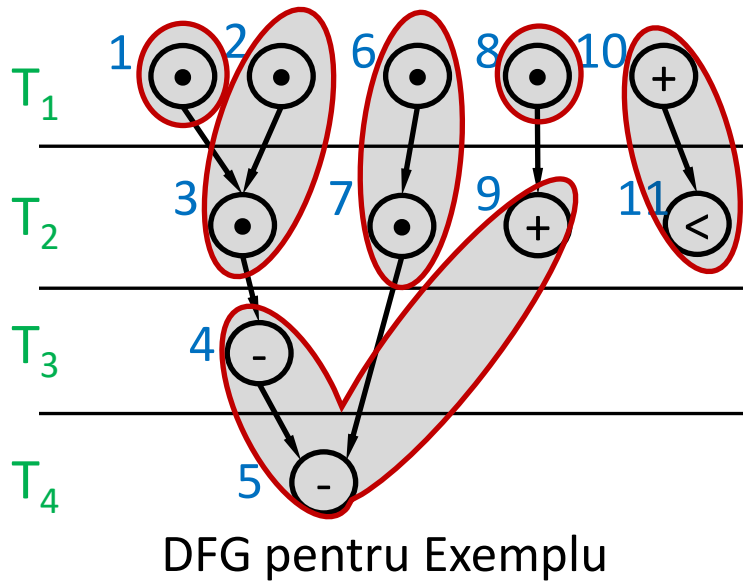
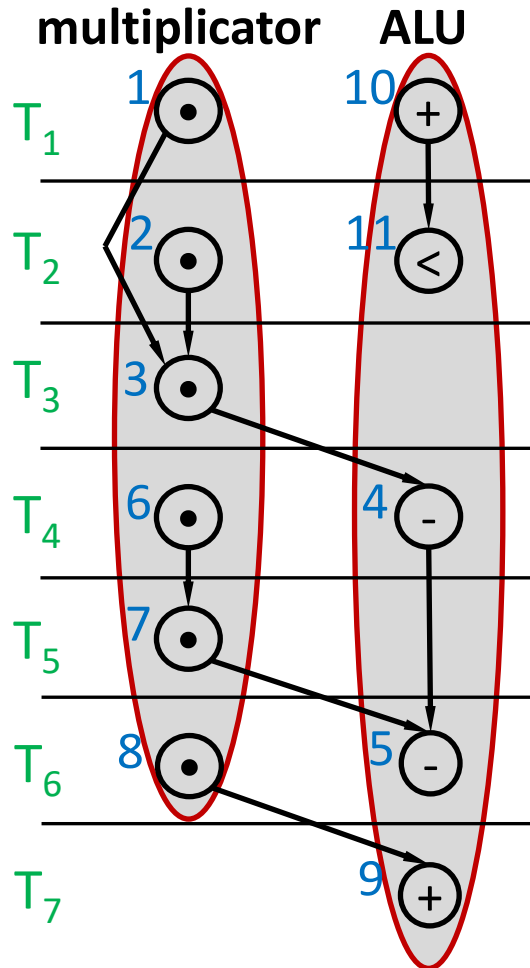


Diagrama tranzițiilor FSM
pentru unitatea de control

2.2 Sinteza la nivel înalt (HLS)

5.B Sinteza unității de control – la varianta **cu constrângeri**

- avem la dispoziție 1 multiplicator (•) și 1 ALU (+, -, <).



DFG cu 1 multiplicator și 1 ALU

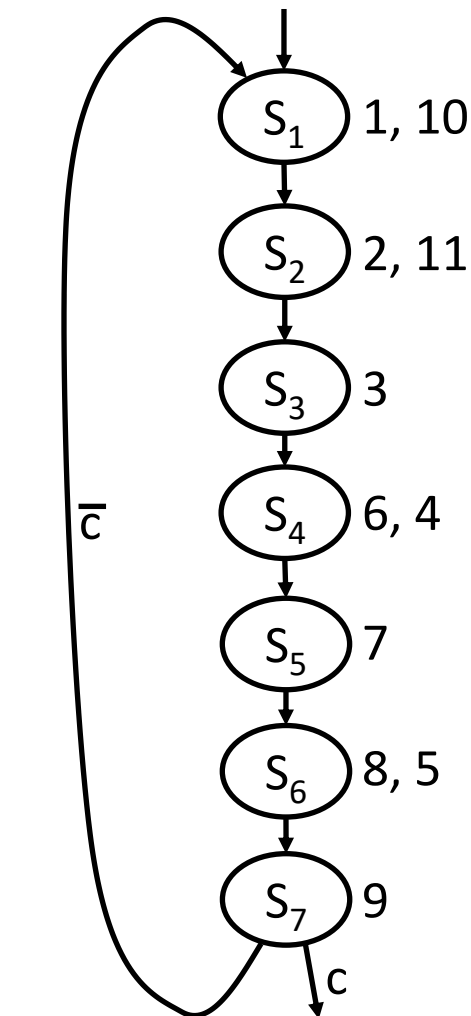
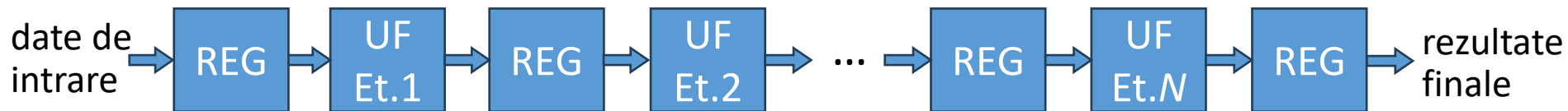


Diagrama tranzițiilor FSM

2.2 Sinteza la nivel înalt (HLS)

2.2.4 Sinteza circuitelor în manieră **pipeline**

- Tehnica pipeline (linie de asamblare) crește productivitatea (throughput).
- Strategia:
 1. Circuitul este **partiționat** în etaje de execuție concurente care, la fiecare impuls, realizează o operație pe date distincte și furnizează rezultatele la etajele următoare;
 2. Între etajele de execuție se amplasează registre de **stocare** a rezultatelor de la etajul anterior, pentru a le **furniza** către etajul următor.



2.2 Sinteza la nivel înalt (HLS)

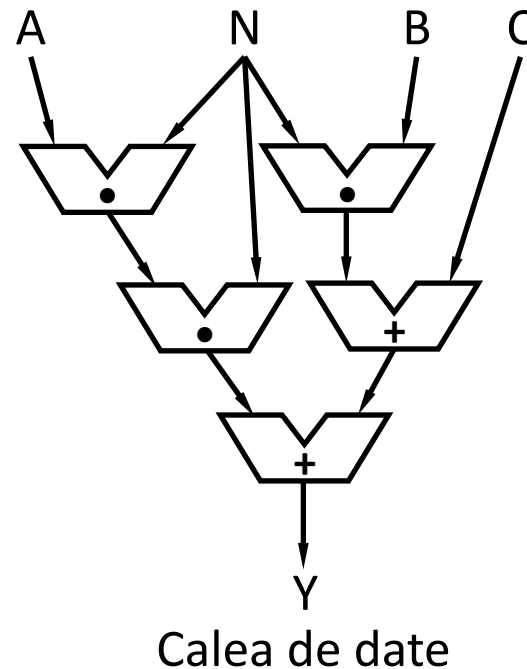
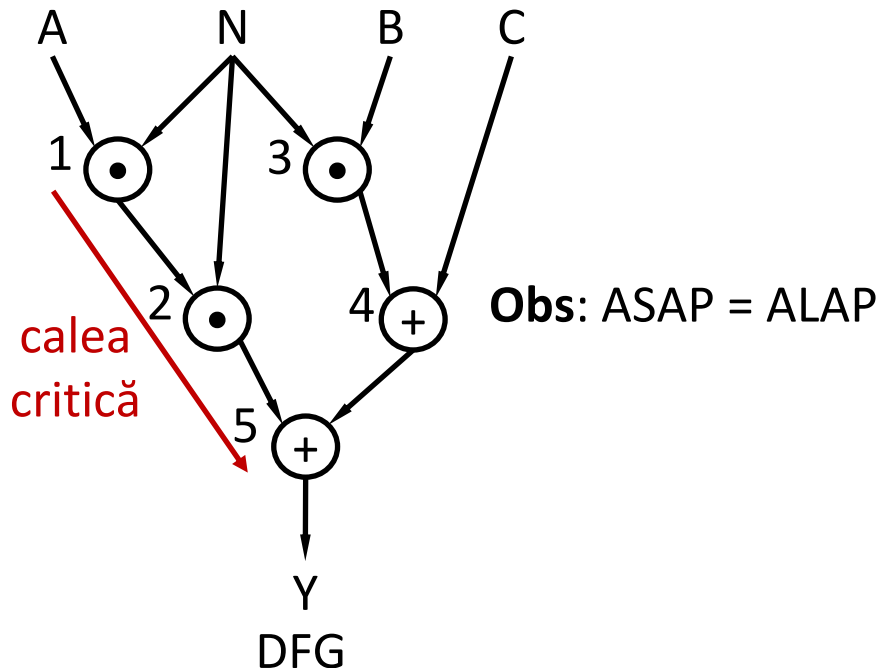
2.2.5 Tehnici de proiectare

- **Ciclu unic (single-cycle):**
 - **Scop:** latență scăzută;
 - **Resurse:** unități funcționale (UF) combinaționale, fără limitare a numărului acestora;
 - **Comunicație:** directă între UF-uri (punct la punct);
 - **Frecvența:** definită de calea critică compusă din UF conectate serial.
- **Linie de asamblare (pipeline):**
 - **Scop:** creșterea productivității (lățimii de bandă) și a frecvenței;
 - **Resurse:** UF combinaționale și registre între etaje;
 - **Comunicație:** prin registrele dintre etaje. Etajele sunt izolate între ele și procesează în paralel diverse faze de execuție;
 - **Frecvența:** definită de UF cea mai lentă.
- **Multi-ciclu (multi-cycle):**
 - **Scop:** reducerea resurselor cu menținerea unei frecvențe crescute;
 - **Resurse:** UF combinaționale în număr limitat și registre pentru rezultate temporare;
 - **Comunicație:** prin registre pentru rezultate temporare, MUX-uri;
 - **Frecvența:** definită de UF cea mai lentă.

2.2 Sinteza la nivel înalt (HLS)

Ex₂: $Y = AN^2 + BN + C \Leftrightarrow (AN + B)N + C \Rightarrow$ 2 variante de implementare

Implementare cu **ciclu unic**, varianta V₁ ($A \bullet N$, $B \bullet N$, $(A \bullet N) \bullet N$, $B \bullet N + C$, $(A \bullet N) \bullet N + B \bullet N + C$)



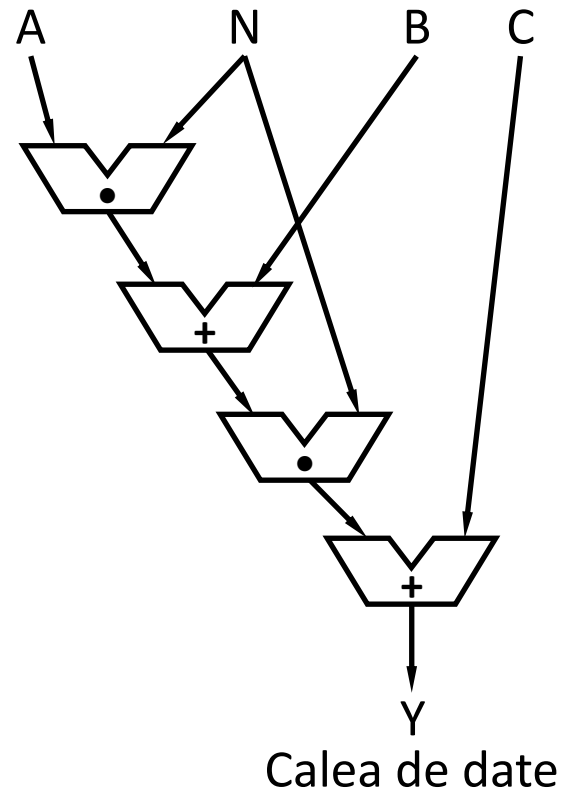
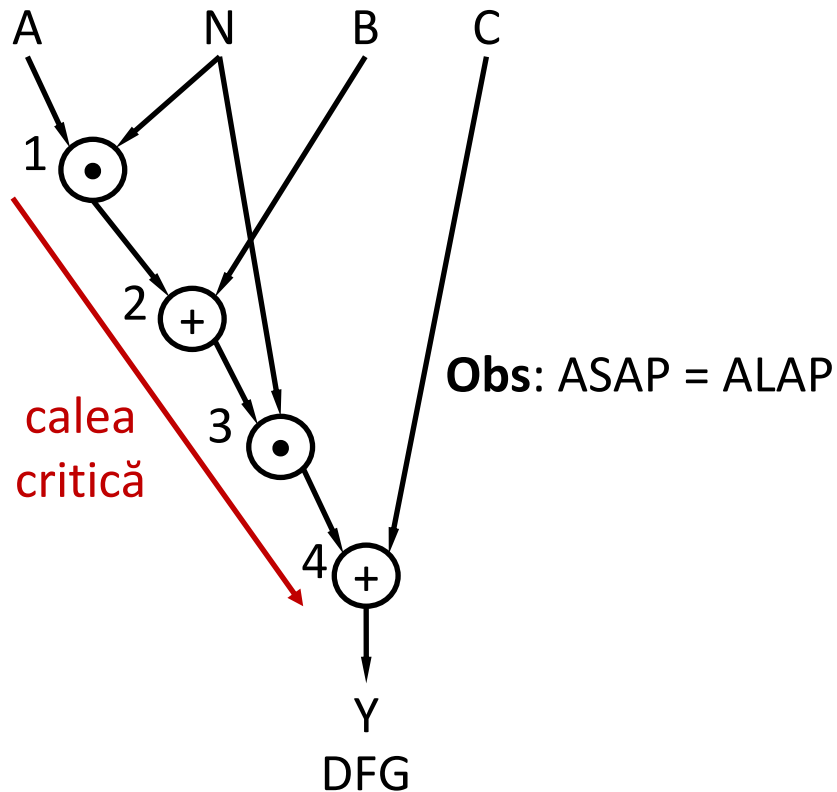
Resurse: 3 înmulțitoare + 2 ALU.

Calea critică: 1, 2, 5 $\Rightarrow$ **Timp de răspuns** (latența): 2 înmulțiri + 1 adunare.

Execuție: controlată de fluxul de date. UF și conexiunile definesc comportamentul. Schimbarea datelor de intrare schimbă rezultatul. **Nu necesită unitate de control.**

2.2 Sinteza la nivel înalt (HLS)

Implementare cu **ciclu unic**, varianta V_2 ($A \bullet N$, $A \bullet N + B$, $(A \bullet N + B) \bullet N$, $(A \bullet N + B) \bullet N + C$)

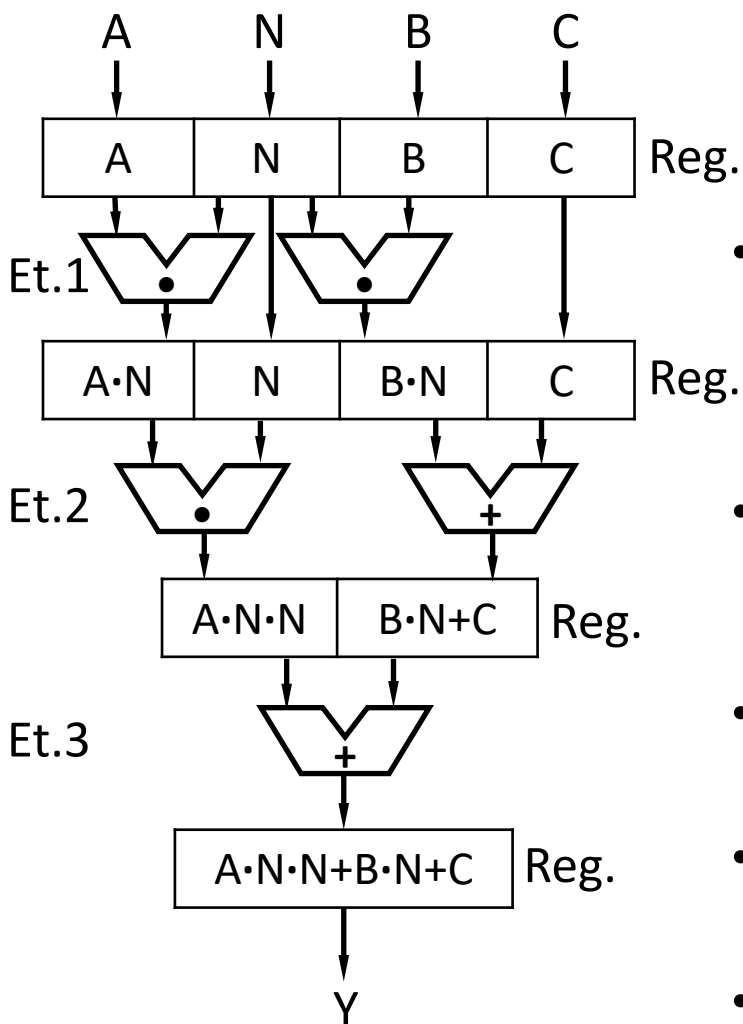


Resurse: 2 înmulțitoare + 2 ALU.

Calea critică: 1, 2, 3, 4 => **Timp de răspuns** (latența): 2 înmulțiri + 2 adunări.

Execuție: controlată de fluxul de date. UF și conexiunile definesc comportamentul. Schimbarea datelor de intrare schimbă rezultatul. **Nu necesită unitate de control.**

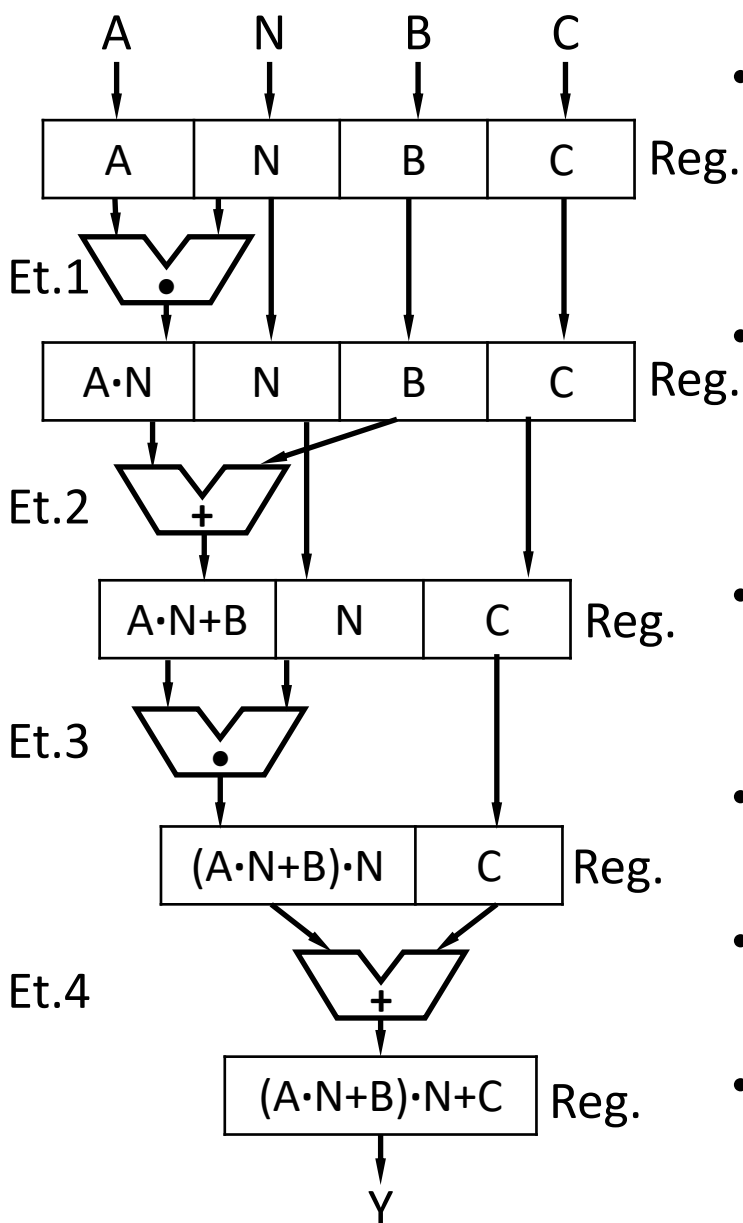
Implementare **pipeline**, varianta V₁



Calea de date (3 etaje pipeline)

- **Productivitate** = 1 rezultat / perioada ceasului $\Rightarrow$ depinde de frecvența ceasului, definită de etajul cel mai lent.
 $\Delta \Leftrightarrow$ Întârziere(delay)
 ▪ Dacă $\Delta_{MUL} = \Delta_{ADD} \Rightarrow$ 3 **etaje balansate**.
- La fiecare ciclu de ceas:
 - Intrare: o nouă pereche de valori (A, N, B, C);
 - Ieșire: rezultatul Y, după umplerea pipeline-ului.
- **Latența** = timpul de propagare prin pipeline = suma întârzierilor prin UF și registre.
 - Mai mare ca la ciclu unic, datorită registrelor.
- **Resurse:** mai multe ca la ciclu unic, datorită registrelor.
- Politica **semnalului de ceas**: impusă de registre, ex. sincronizare pe frontul ascendent.
- **Execuție:** controlată de fluxul de date. UF și conexiunile definesc comportamentul. Schimbarea datelor de intrare schimbă rezultatul. **Nu necesită unitate de control.**

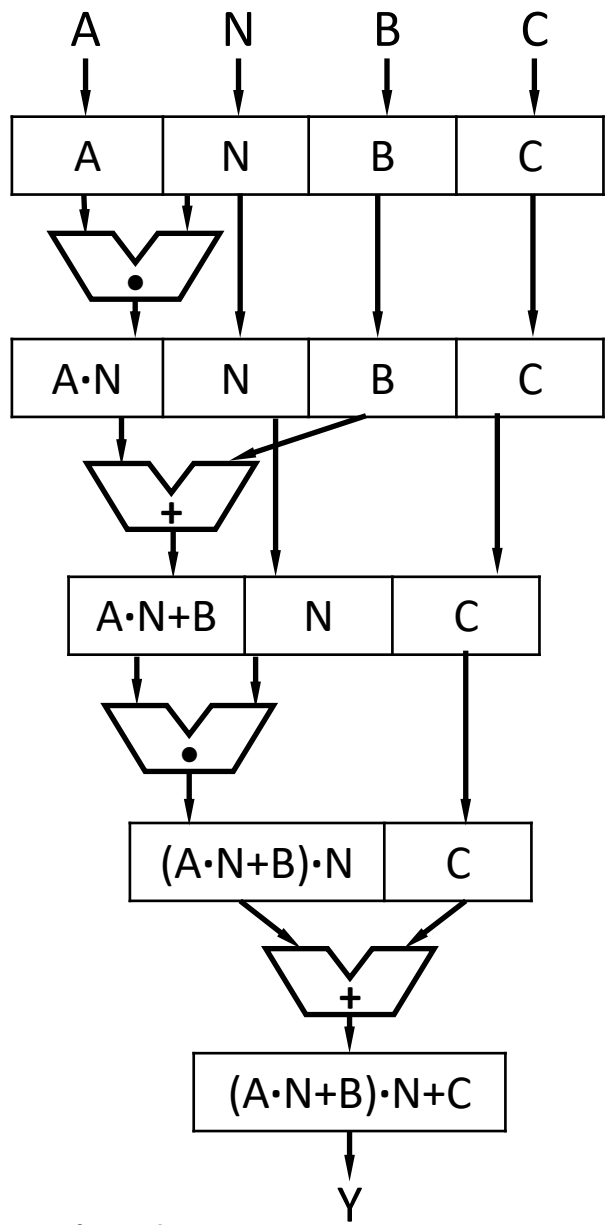
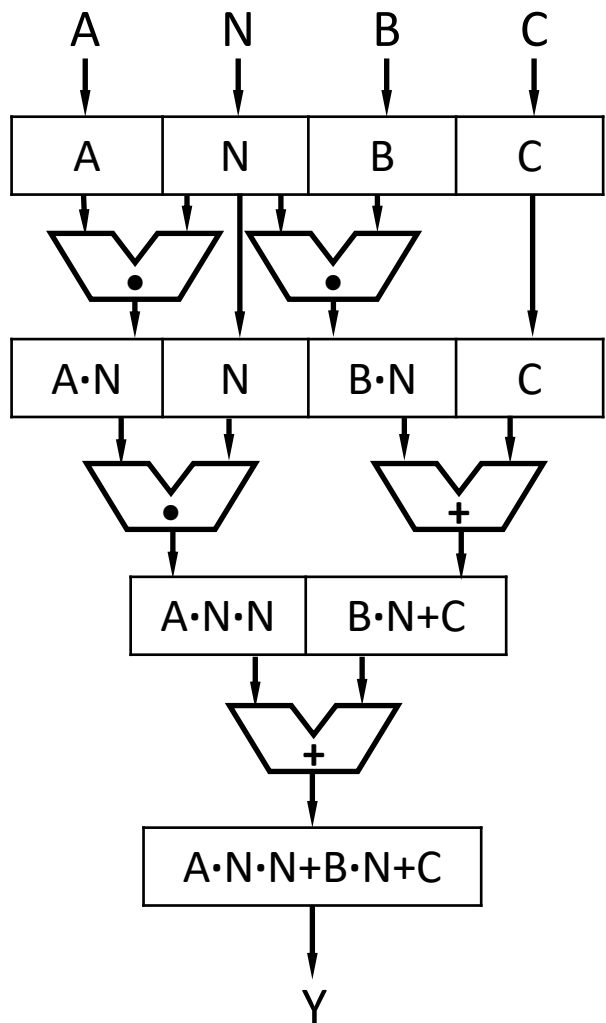
Implementare pipeline, varianta V₂



Calea de date (4 etaje pipeline)

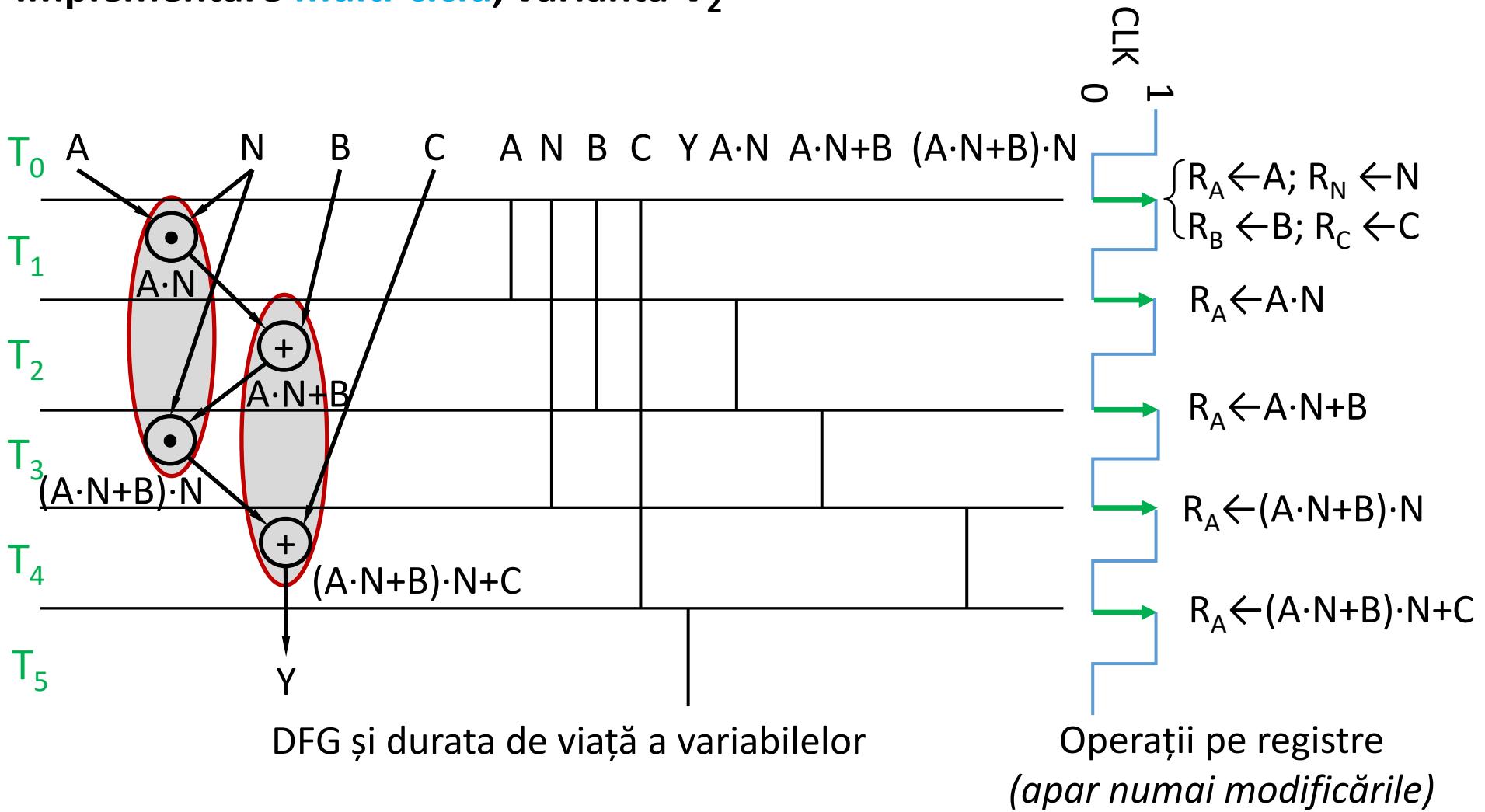
- **Productivitate** = 1 rezultat / perioada ceasului => depinde de frecvența ceasului, definită de etajul cel mai lent.
 $\Delta \Leftrightarrow$ întârziere (delay)
 ▪ Dacă $\Delta_{MUL} = \Delta_{ADD} \Rightarrow$ 4 **etaje balansate**.
- La fiecare ciclu de ceas:
 - Intrare: o nouă pereche de valori (A, N, B, C);
 - Ieșire: rezultatul Y, după umplerea pipeline-ului.
- **Latența** = timpul de propagare prin pipeline = suma întârzierilor prin UF și registre.
 - Mai mare ca la ciclu unic, datorită registrelor.
- **Resurse:** mai multe ca la ciclu unic, datorită registrelor.
- Politica **semnalului de ceas**: impusă de registre, ex. sincronizare pe frontul ascendent.
- **Execuție:** controlată de fluxul de date. UF și conexiunile definesc comportamentul. Schimbarea datelor de intrare schimbă rezultatul. **Nu necesită unitate de control.**

Implementare pipeline, comparație V_1 vs. V_2

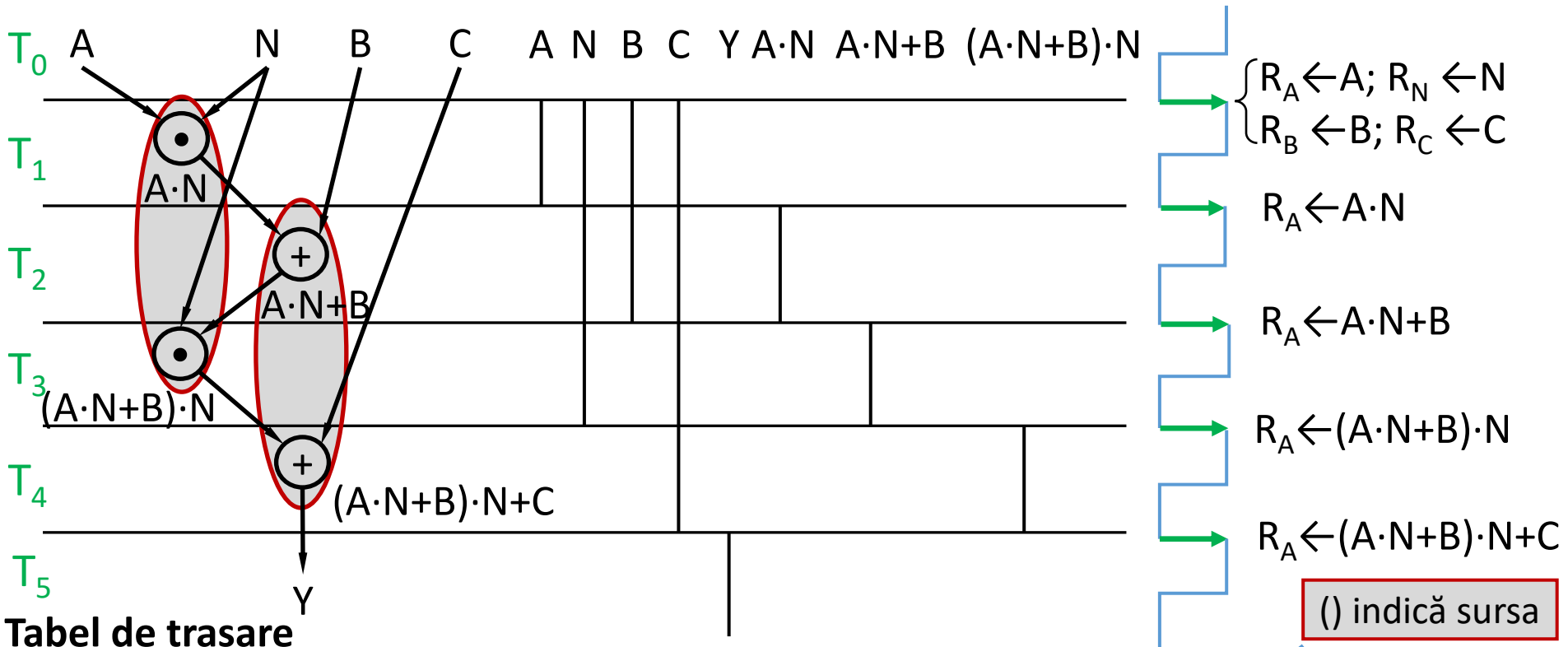


- **Latenta:** V_1 (3 etaje) < V_2 (4 etaje).
- **Productivitate:** $V_1 = V_2$ (aceeași frecvență de ceas).
- **Resurse:** V_1 are un înmulțitor în plus, V_2 are un registru în plus.

Implementare multi-ciclu, varianta V₂



- Planificare cu constrângere: 1 înmulțitor + 1 ALU (realizează suma).
 - înmulțitor: in_1 (intrare stânga), in_2 (intrare dreapta);
 - ALU: in_1 (intrare stânga), in_2 (intrare dreapta).
- Pentru memorarea variabilelor și a valorilor temporare se folosesc registre.
- Conexiunea $UF \leftrightarrow$ registre: directă sau cu MUX.



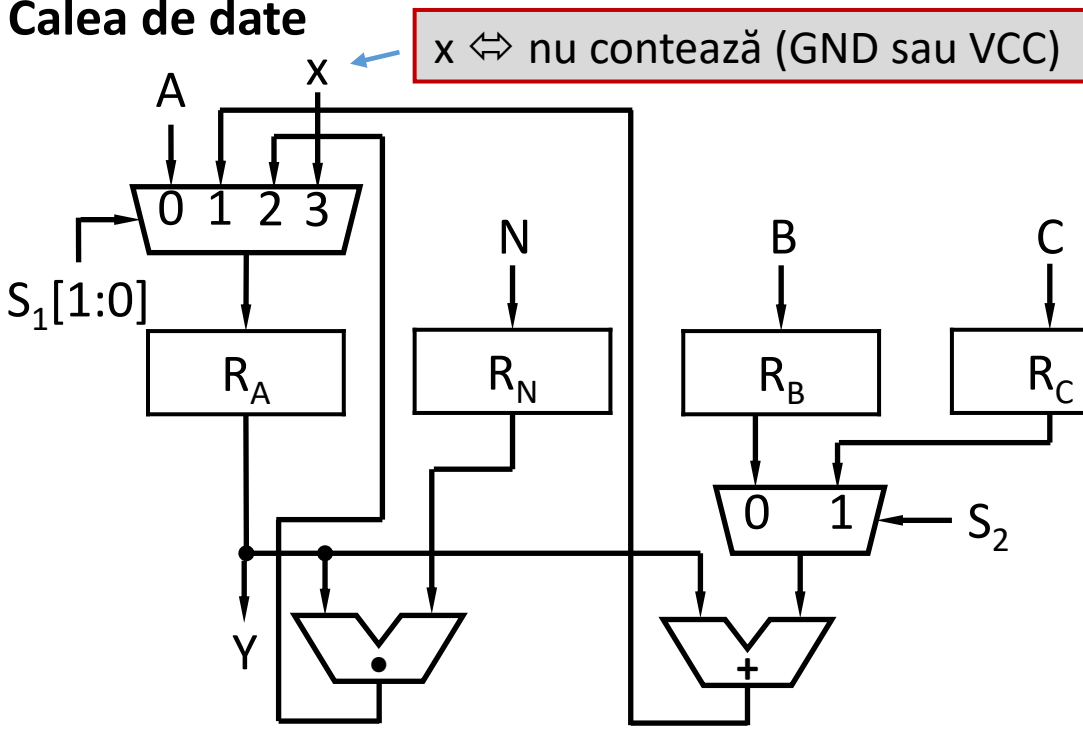
Tabel de trasare

Resursa/portul		T_0	T_1	T_2	T_3	T_4
înmulțitor	in_0		$A (R_A)$		$A \cdot N + B (R_A)$	
înmulțitor	in_1		$N (R_N)$		$N (R_N)$	
înmulțitor	out		$A \cdot N$		$(A \cdot N + B) \cdot N$	
sumator	in_0		<i>(gol = nu contează)</i>	$A \cdot N (R_A)$		$(A \cdot N + B) \cdot N (R_A)$
sumator	in_1			$B (R_B)$		$C (R_C)$
sumator	out			$A \cdot N + B$		$(A \cdot N + B) \cdot N + C$
R_A	in	A	$A \cdot N$ (înmulțitor)	$A \cdot N + B$ (sumator)	$(A \cdot N + B) \cdot N$ (înmulțitor)	$(A \cdot N + B) \cdot N + C$ (sumator)
R_N	in	N	N	N		
R_B	in	B	B			
R_C	in	C	C	C	C	

Tabel de trasare

Resursa/portul		T_0	T_1	T_2	T_3	T_4
înmulțitor	in_0		$A (R_A)$		$A \cdot N + B (R_A)$	
înmulțitor	in_1		$N (R_N)$		$N (R_N)$	
înmulțitor	out		$A \cdot N$		$(A \cdot N + B) \cdot N$	
sumator	in_0		<i>(gol = nu contează)</i>	$A \cdot N (R_A)$		$(A \cdot N + B) \cdot N (R_A)$
sumator	in_1			$B (R_B)$		$C (R_C)$
sumator	out			$A \cdot N + B$		$(A \cdot N + B) \cdot N + C$
R_A	in	A	$A \cdot N$ (înmulțitor)	$A \cdot N + B$ (sumator)	$(A \cdot N + B) \cdot N$ (înmulțitor)	$(A \cdot N + B) \cdot N + C$ (sumator)
R_N	in	N	N	N		
R_B	in	B	B			
R_C	in	C	C	C	C	

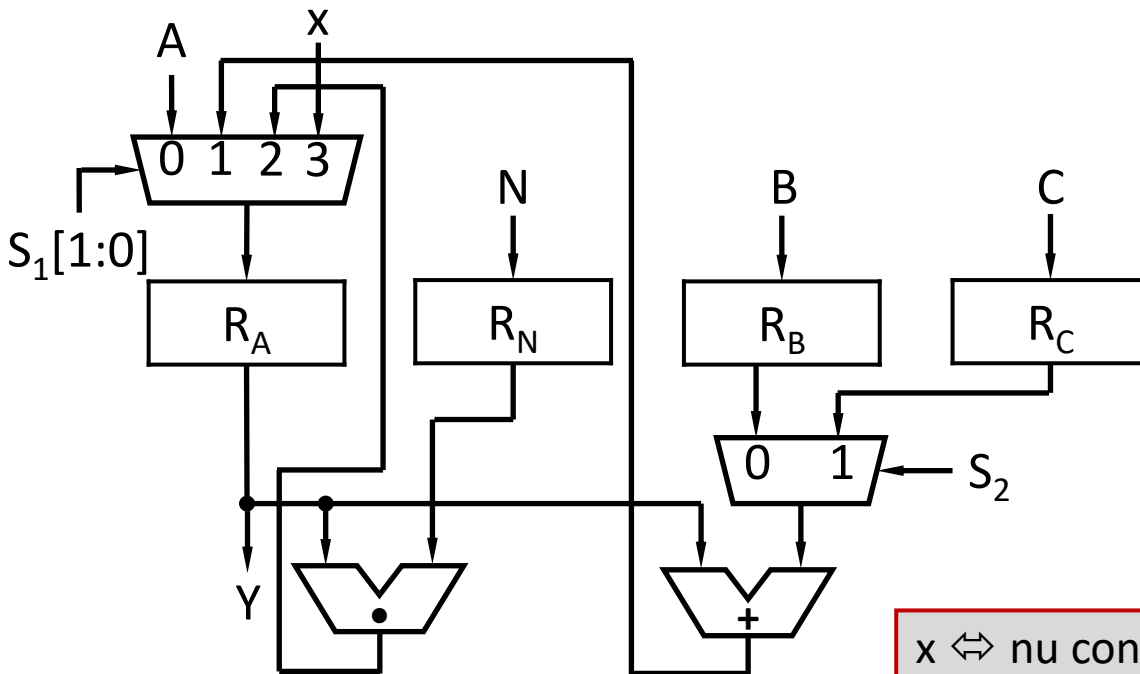
Calea de date



Conexiuni:

- înmulțitor in_0 : conexiune la R_A ;
- înmulțitor in_1 : conexiune la R_N ;
- sumator in_0 : conexiune la R_A ;
- sumator in_1 : MUX 2:1;
- R_A in: MUX 4:1;
- R_N in: conexiune la N;
- R_B in: conexiune la B;
- R_C in: conexiune la C;
- Y: conexiune la R_A ;
- S_1 , S_2 : generate de unitatea de control.

Calea de date



$x \Leftrightarrow$ nu contează (0 sau 1)

Descrivere RTL

$$T_0: R_A \leftarrow A; R_N \leftarrow N; R_B \leftarrow B; R_C \leftarrow C$$
$$T_1: R_A \leftarrow R_A \cdot R_N$$
$$T_2: R_A \leftarrow R_A + R_B$$
$$T_3: R_A \leftarrow R_A \cdot R_N$$
$$T_4: R_A \leftarrow R_A + R_C$$

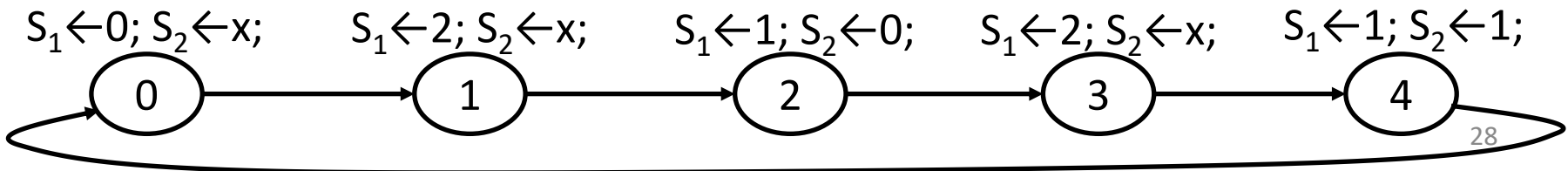
Control (MUX)

$$S_1 \leftarrow 0; S_2 \leftarrow x$$
$$S_1 \leftarrow 2; S_2 \leftarrow x$$
$$S_1 \leftarrow 1; S_2 \leftarrow 0$$
$$S_1 \leftarrow 2; S_2 \leftarrow x$$
$$S_1 \leftarrow 1; S_2 \leftarrow 1$$

Trasare registre (valorile lor)

$$(R_A; R_N; R_B; R_C) = (Y; N; B; C)_{\text{anterior}}$$
$$(R_A; R_N; R_B; R_C) = (A; N; B; C)_{\text{current}}$$
$$R_A = A \cdot N; R_N = N; R_B = B; R_C = C$$
$$R_A = A \cdot N + B; R_N = N; R_B = B; R_C = C$$
$$R_A = (A \cdot N + B) \cdot N; R_N = N; R_B = B; R_C = C$$

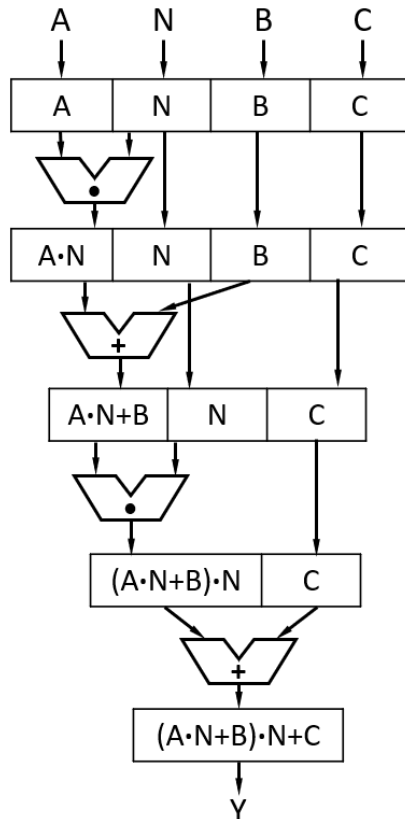
Unitatea de control – FSM cu 5 stări (0, 1, 2, 3, 4) – la multi-ciclu este necesară UC



Pipeline vs. Multi-ciclu

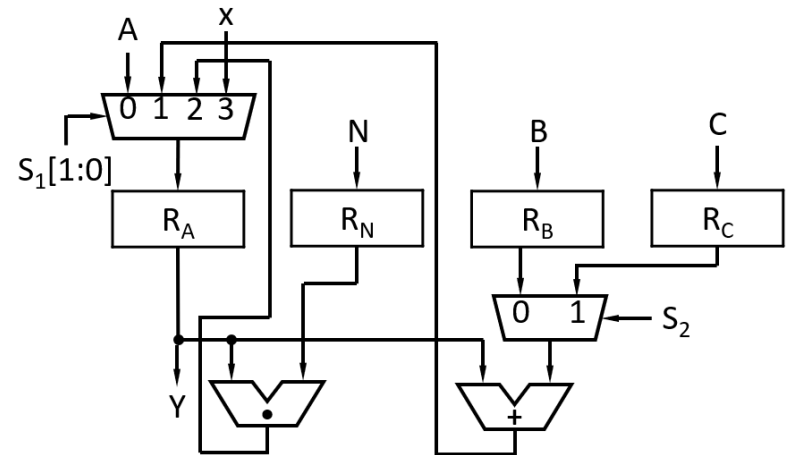
Pipeline

- Inserare pereche (A, N, B, C) la fiecare ciclu;
- Extragere rezultat la fiecare ciclu după umplere – productivitate mare;
- Frecvență mare.
- Număr mare de registre.
- Număr mare de UF.



Multi-ciclu

- Inserare pereche (A, N, B, C) la fiecare al 5-lea ciclu (ciclul T_0);
- Extragere rezultat (Y) la fiecare al 5-lea ciclu (ciclul T_0) – productivitate redusă;
- Frecvență mare.
- Reduce numărul de registre la minim.
- Reduce numărul de UF la minim.



Problemă propusă

Pentru următoarea ecuație: $W = A \cdot (B \cdot M + Z) - C \cdot (A \cdot P + M)$ se presupune:

- a) Planificare fără constrângeri.
- b) Planificare cu 1 ALU (sumator/scăzător) și 1 unitate de multiplicare.

Pentru variantele a) și b) arătați DFG, timpul de viață al variabilelor, numărul minim de registre necesare, căile de date și unitățile de control corespunzătoare la proiectare multi-ciclu.

Referințe

- [1] F. Oniga, “De la bit la procesor. Introducere în arhitectura calculatoarelor”, Cap. 8.1, Ed. U.T. Press, 2019, ISBN 978-606-737-366-0.
- [2] P. Eles, Z. Peng, “Lecture 1: Introduction”, System Synthesis of Digital Systems, March 2000, Online: <http://www.ida.liu.se/~petel/SysSyn/lect1.frm.pdf>
- [3] M. Fingeroff, “High-Level Synthesis Blue Book”. Mentor Graphics Corporation, 2010.
- [4] P. Coussy, D. Gajski, M. Meredith, A. Takach, “An introduction to high-level synthesis”, IEEE Design & Test of Computers, Vol. 26, Issue 4, 2009, pp. 8–17.